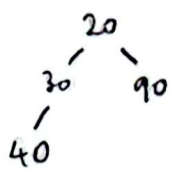
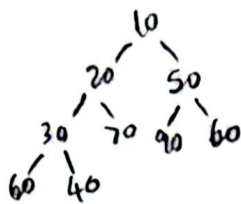


a) Tree #1



Tree #2



b) for $i \leftarrow 1$ to $n/2$:
 if $A[i] > A[2 \cdot i]$ or $(2 \cdot i + 1 \leq n$ and $A[2 \cdot i + 1] > A[i])$:
 return false
 return true.

This algorithm checks if any element of the tree exceeds the value of one of its children; if this is true, then A doesn't represent a min heap. (So return false) else return true.

The algorithm is $O(n)$ time complexity (runs an $O(1)$ loop $n/2$ times) and $O(1)$ space complexity.

c) min heap sorted copy of
 $B[1 \dots k] \leftarrow A[1 \dots k]$

~~sort $B[1 \dots k]$~~

for $i \leftarrow k+1$ to n :

if $A[i] > B[i]$:

$B.REMOVEMIN()$

$B.INSERT(A[i])$

return B // B contains k largest #s

This algorithm maintains a min heap B which stores the k running largest numbers by deleting the min and inserting $A[i]$ whenever $A[i] > B[i]$, as $B[i]$ is the minimum of B by definition of min heap.

The time complexity is $O(k \log k + (n-k) \log k)$, as $k \log k$ comes from sorting $A[1 \dots k]$ and $(n-k) \log k$ comes from potentially $n-k$ inserts each of which is $\log k$ complexity.

The space complexity is $O(k)$, since that's how much space B takes up.

d) stack route $\leftarrow \{1\}$

minentries: array list with capacity n
 while route.size > 0 do

$t \leftarrow \text{route.top}()$

route.pop()

if $A[t].key \leq k$:

minentries.add($A[t].entry$)

if $2 \cdot t + 1 \leq n$:

route.push($2 \cdot t + 1$) // right child

if $2 \cdot t \leq n$:

route.push($2 \cdot t$) // left child

return minentries

This algorithm does a preorder traversal of the heap tree, except not traversing children of nodes whose keys are larger than k . Thus at most $\frac{1}{2}$ of the nodes accessed are not returned. The (time) complexity is thus $O(n)$, where r entries are returned, and space complexity is $O(n)$ due to the array list.

2a) create hash table B

for $i \leftarrow 1$ to n

if $B.get(s-A[i]) \neq \text{null}$ // assuming get returns null if key not found
return $(i, B.get(s-A[i]))$

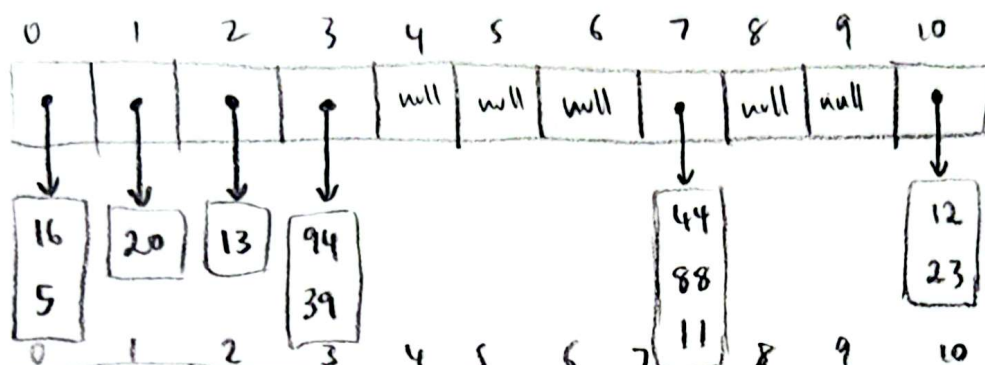
B.put($(A[i], i)$)

return null

If an i, j pair exists then, supposing $i > j$, when the loop reaches i
We get that $B.get(s-A[i])$ returns some index $k < i$ such that $A[k] = A[j]$
So the existence of j guarantees the existence of this index, which is why the algorithm works.

Time complexity is $O(n)$ on average because get, put are $O(1)$ on average,
and space complexity is $O(n)$ due to hash table B potentially storing all $(A[i], i)$ pairs.

2b) (a)



(b)

0	1	2	3	4	5	6	7	8	9	10
23	20	13	94	39	16	5	44	88	11	12

(c)

0	1	2	3	4	5	6	7	8	9	10
23	20	13	94	39	11	null	44	88	16	12

The remaining number 5, maps to 0, so quadratic probing allows us to place it in 0, 1, 4, 9, 5, or 3, all of which are taken.